

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Everything is working!")
print("OpenCV:", cv2.__version__)
```

Everything is working!
OpenCV: 5.0.0

```
In [2]: video_path = "video.mp4"

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully!")

    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    print("FPS:", fps)
    print("Width:", width)
    print("Height:", height)
    print("Total Frames:", total_frames)
```

Video opened successfully!
FPS: 24.0
Width: 1280
Height: 720
Total Frames: 240

```
In [3]: ret, old_frame = cap.read()

if ret:
    print("First frame successfully read.")

    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(old_frame, cv2.COLOR_BGR2RGB))
    plt.title("First Frame")
    plt.axis("off")
    plt.show()

else:
    print("Could not read first frame.")
```

First frame successfully read.

First Frame



```
In [4]: feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

if p0 is not None:
    print("Feature points detected:", len(p0))

    feature_frame = old_frame.copy()

    for point in p0:
        x, y = point.ravel()
        cv2.circle(
            feature_frame,
            (int(x), int(y)),
            5,
            (0, 255, 0),
            -1
        )

    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(feature_frame, cv2.COLOR_BGR2RGB))
    plt.title("Shi-Tomasi Feature Points")
    plt.axis("off")
    plt.show()
```

```

else:
    print("No feature points detected.")

```

Feature points detected: 85

Shi-Tomasi Feature Points



```

In [5]: lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(
            cv2.TERM_CRITERIA_EPS |
            cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    )

    print("Lucas-Kanade parameters initialized.")

```

Lucas-Kanade parameters initialized.

```

In [6]: cap.release()

        cap = cv2.VideoCapture(video_path)

        ret, old_frame = cap.read()

        if not ret:
            print("Could not read video.")
        else:
            old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

            p0 = cv2.goodFeaturesToTrack(
                old_gray,
                mask=None,

```

```

        **feature_params
    )

    trajectory_mask = np.zeros_like(old_frame)

    frames_to_show = []
    motion_values = []

    frame_count = 0
    max_frames = 100

    while frame_count < max_frames:

        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        if p0 is not None and len(p0) > 0:

            p1, status, error = cv2.calcOpticalFlowPyrLK(
                old_gray,
                frame_gray,
                p0,
                None,
                **lk_params
            )

            if p1 is not None:

                good_new = p1[status == 1]
                good_old = p0[status == 1]

                displacements = []

                for new, old in zip(good_new, good_old):

                    x_new, y_new = new.ravel()
                    x_old, y_old = old.ravel()

                    # Draw trajectory
                    cv2.line(
                        trajectory_mask,
                        (int(x_old), int(y_old)),
                        (int(x_new), int(y_new)),
                        (0, 255, 0),
                        2
                    )

                    # Draw current feature point
                    cv2.circle(
                        frame,
                        (int(x_new), int(y_new)),
                        4,

```

```

        (0, 0, 255),
        -1
    )

    # Calculate displacement
    dx = x_new - x_old
    dy = y_new - y_old

    magnitude = np.sqrt(dx**2 + dy**2)

    displacements.append(magnitude)

    # Combine frame and trajectories
    output = cv2.add(frame, trajectory_mask)

    if len(displacements) > 0:
        avg_displacement = np.mean(displacements)
        motion_values.append(avg_displacement)

    # Save selected frames
    if frame_count in [10, 30, 50, 70, 90]:
        frames_to_show.append(
            cv2.cvtColor(output, cv2.COLOR_BGR2RGB)
        )

    p0 = good_new.reshape(-1, 1, 2)

    else:
        break

    old_gray = frame_gray.copy()

    frame_count += 1

cap.release()

print("Frames processed:", frame_count)

```

Frames processed: 100

```

In [7]: for i, image in enumerate(frames_to_show):

    plt.figure(figsize=(10, 6))

    plt.imshow(image)

    plt.title(
        f"Lucas-Kanade Optical Flow - Frame {i+1}"
    )

    plt.axis("off")

    plt.show()

```

Lucas-Kanade Optical Flow - Frame 1



Lucas-Kanade Optical Flow - Frame 2



Lucas-Kanade Optical Flow - Frame 3



Lucas-Kanade Optical Flow - Frame 4



Lucas-Kanade Optical Flow - Frame 5



Step 7 — Calculate Motion Magnitude and Direction

Optical Flow vector:

$$(dx, dy)$$

Motion magnitude:

$$M = \sqrt{dx^2 + dy^2}$$

Direction:

$$\theta = \tan^{-1} \left(\frac{dy}{dx} \right)$$

```
In [8]: if len(motion_values) > 0:
        average_displacement = np.mean(motion_values)
        print(
            "Average Displacement:",
            round(average_displacement, 2),
            "pixels/frame"
        )
        if fps > 0:
            speed = average_displacement * fps
            print(
```



```

        "Estimated Average Speed:",
        round(speed, 2),
        "pixels/second"
    )

else:

    print("No motion detected.")

```

Average Displacement: 1.31 pixels/frame
 Estimated Average Speed: 31.53 pixels/second

Step 8 — Farneback Dense Optical Flow

Ab Lucas-Kanade ke instead Farneback Dense Optical Flow use karenge.

Lucas-Kanade selected points track karta hai.

Farneback approximately dense motion field calculate karta hai.

Code Cell 9

```

In [9]: cap = cv2.VideoCapture(video_path)

ret, frame1 = cap.read()

if not ret:
    print("Could not read video.")
else:

    previous_gray = cv2.cvtColor(
        frame1,
        cv2.COLOR_BGR2GRAY
    )

    dense_flow_frames = []

    frame_count = 0
    max_frames = 50

    while frame_count < max_frames:

        ret, frame2 = cap.read()

        if not ret:
            break

        next_gray = cv2.cvtColor(
            frame2,
            cv2.COLOR_BGR2GRAY
        )

        flow = cv2.calcOpticalFlowFarneback(
            previous_gray,
            next_gray,
            None,

```

```

        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    hsv = np.zeros_like(frame2)

    hsv[..., 1] = 255

    hsv[..., 0] = (
        angle * 180 / np.pi / 2
    )

    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    optical_flow = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2RGB
    )

    if frame_count in [10, 20, 30, 40]:
        dense_flow_frames.append(
            optical_flow
        )

    previous_gray = next_gray

    frame_count += 1

cap.release()

print(
    "Farneback frames processed:",
    frame_count
)

```

Farneback frames processed: 50

```

In [10]: for i, image in enumerate(dense_flow_frames):

    plt.figure(figsize=(10, 6))

```

```
plt.imshow(image)\n\nplt.title(\n    f"Farneback Dense Optical Flow - Result {i+1}"\n)\n\nplt.axis("off")\n\nplt.show()
```

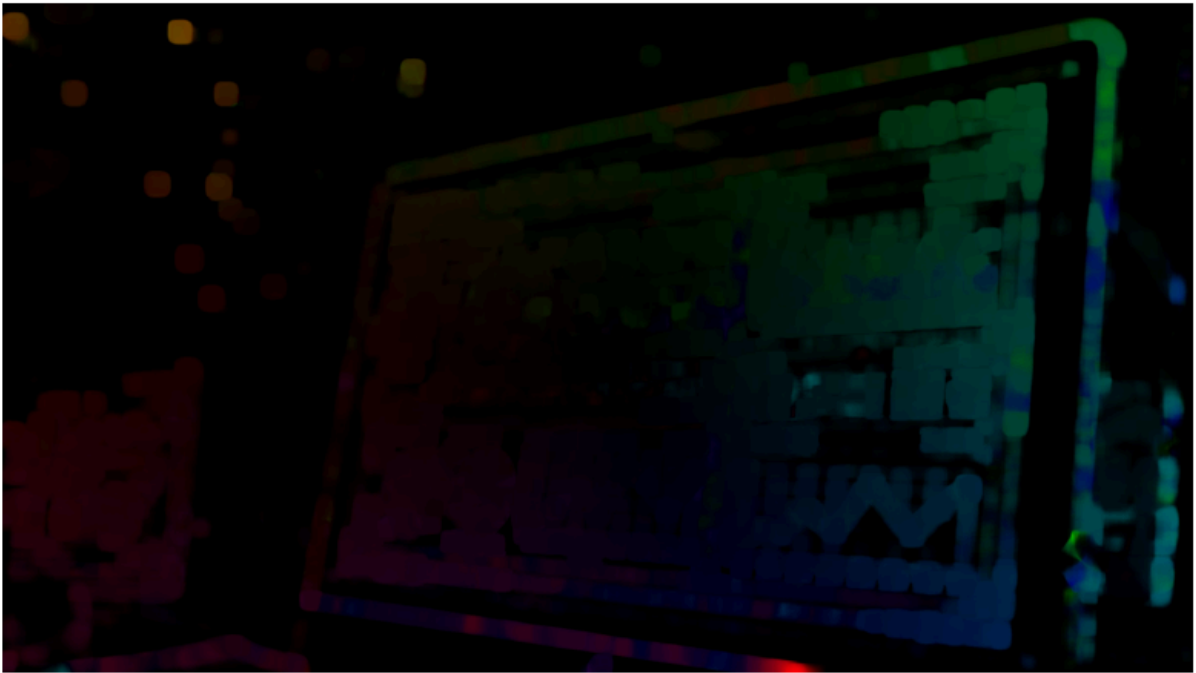
Farneback Dense Optical Flow - Result 1



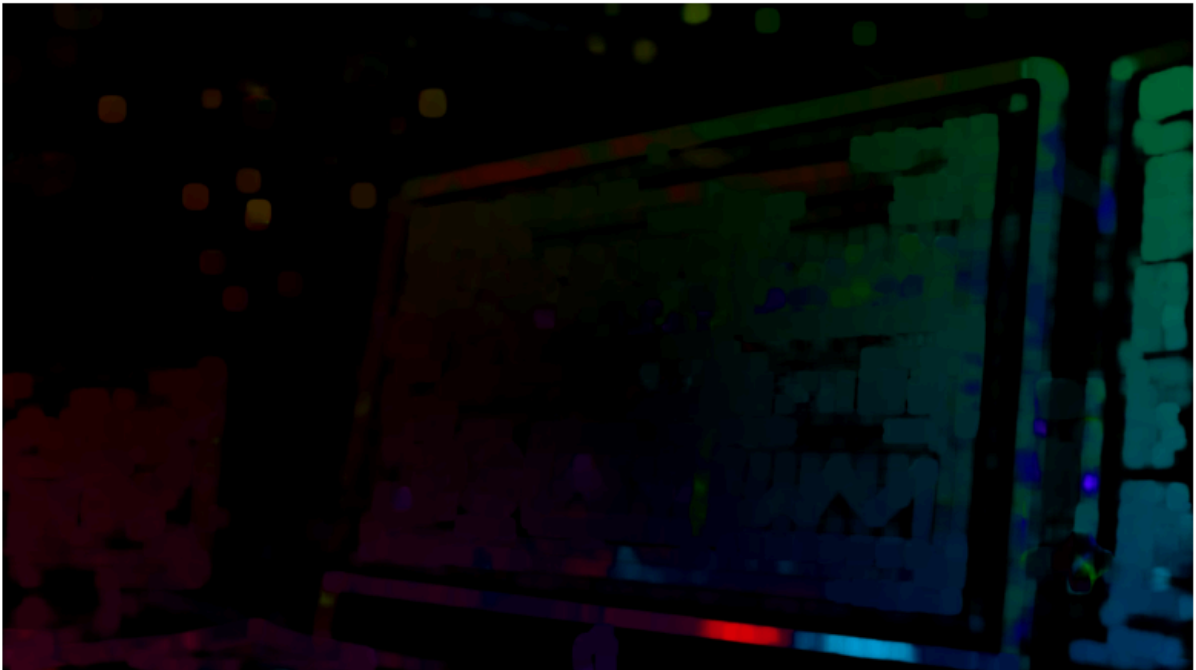
Farneback Dense Optical Flow - Result 2



Farneback Dense Optical Flow - Result 3



Farneback Dense Optical Flow - Result 4



Step 10 — Compare Lucas-Kanade and Farneback Comparison Table Feature Lucas-Kanade Farneback Type Sparse Optical Flow Dense Optical Flow Tracking Selected feature points Almost every pixel Speed Generally faster Generally slower Computation Lower Higher Output Point trajectories Dense motion field Best use Object/feature tracking Overall motion analysis Feature detection Required Not necessarily required Motion visualization Lines/points Color-based flow field Step 11 — Analyze Tracking Challenges

Experiment me following conditions ko analyze karna hai:

1. Fast-moving object

Fast object ke movement me displacement zyada hota hai.

Problem:

Feature points lose ho sakte hain. Motion blur aa sakta hai. Tracking accuracy decrease ho sakti hai. 2. Illumination changes

Lighting suddenly change hone par pixel intensity change hoti hai.

Isse optical flow estimation affect ho sakta hai.

3. Occlusion

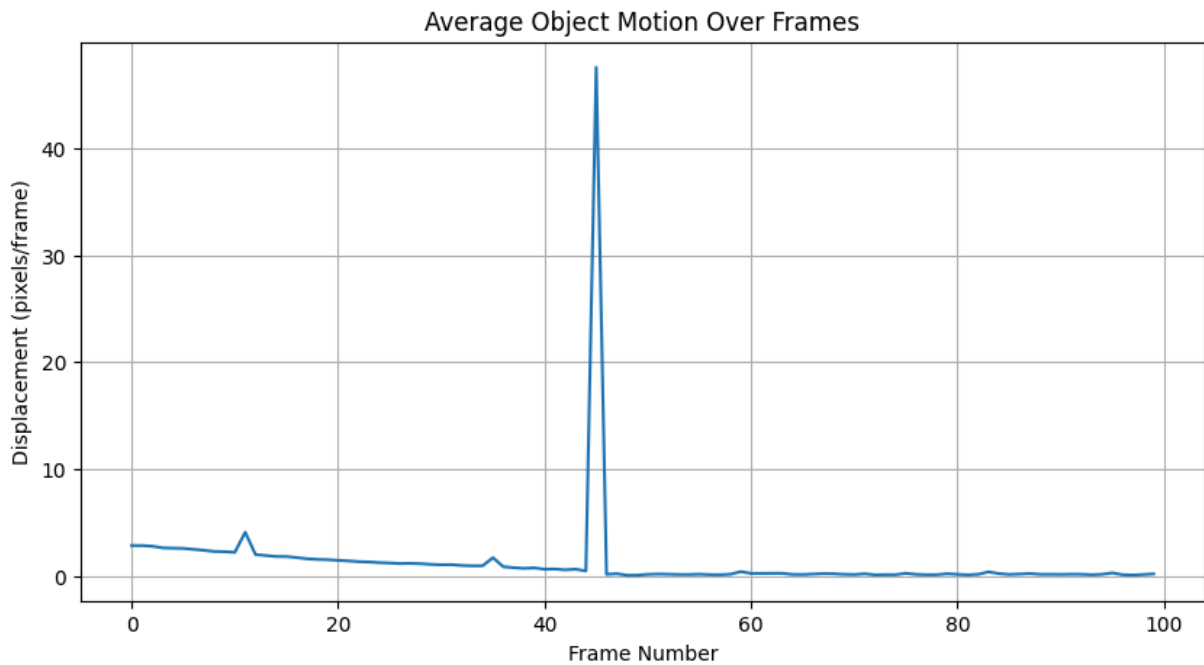
Agar object kisi doosre object ke peeche chala jaye, feature points temporarily disappear ho sakte hain.

4. Camera movement

Agar camera move kare, to frame ka large portion move karta hua appear hota hai.

Isse object motion aur camera motion ko distinguish karna difficult ho sakta hai.

```
In [11]: plt.figure(figsize=(10, 5))  
  
plt.plot(motion_values)  
  
plt.title("Average Object Motion Over Frames")  
plt.xlabel("Frame Number")  
plt.ylabel("Displacement (pixels/frame)")  
  
plt.grid(True)  
plt.show()
```

Observations

Tum practical file me ye observations likh sakti ho:

Shi-Tomasi corner detection successfully detected suitable feature points in the video. Lucas-Kanade Optical Flow tracked the selected feature points between consecutive frames. Motion trajectories were visualized using lines and feature points. Displacement and average motion were calculated in pixels/frame. Farneback Optical Flow generated a dense motion field over the image. Fast-moving objects produced larger optical-flow vectors. Illumination changes and occlusion affected tracking accuracy. Camera movement produced motion across a large portion of the frame. Lucas-Kanade was suitable for sparse feature tracking. Farneback was useful for dense motion analysis. Result

Result: The optical-flow-based object tracking system was successfully implemented using Python and OpenCV. Lucas-Kanade Optical Flow was used for sparse feature tracking and trajectory estimation, while Farneback Dense Optical Flow was used for analyzing motion across the image. Object displacement, motion direction, and average motion speed were estimated from consecutive video frames.

Questions & Answers

1. How does object tracking differ from object detection?

Object Detection identifies the location and class of an object in an image or video frame.

Object Tracking follows the same object across multiple consecutive frames.

Example:

Detection → Find a car. Tracking → Follow that car as it moves through the video. 2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow estimates the apparent motion of pixels between consecutive frames.

The process is:

Frame 1 ↓ Detect feature points ↓ Frame 2 ↓ Calculate displacement ↓ Track points
↓ Repeat

Lucas-Kanade Optical Flow can track selected feature points and generate their trajectories.

3. What is the role of Shi-Tomasi Corner Detection in Lucas-Kanade Optical Flow?

Shi-Tomasi identifies strong and stable corner points in an image.

These points are used as features to track using Lucas-Kanade Optical Flow.

Therefore:

Shi-Tomasi → finds good points

Lucas-Kanade → tracks those points

4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow estimates the movement of pixels between consecutive frames.

It provides:

Motion direction Motion magnitude Object displacement Motion trajectories
Movement patterns

Therefore, it is useful for analyzing dynamic scenes.

5. What challenges arise while tracking fast-moving or partially occluded objects?

Major challenges include:

Motion blur Large displacement Loss of feature points Partial occlusion Illumination changes Low-texture objects Camera movement

These factors can reduce tracking accuracy.

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods. Optical Flow Deep Learning Tracking Uses pixel/feature motion Uses learned features Usually computationally lighter Usually computationally heavier

Does not require large training datasets for basic methods Usually requires training data/models Sensitive to illumination and occlusion Can be more robust depending on model Good for motion estimation Can provide object identity/class information Examples: LK, Farneback Examples: YOLO-based tracking, Deep SORT

7. How can Optical Flow be used in traffic monitoring and autonomous driving?

Optical Flow can estimate movement of:

Vehicles Pedestrians Road objects

Applications include:

Vehicle speed estimation Traffic flow analysis Collision warning Lane movement analysis Pedestrian movement detection Ego-motion estimation 8. Explain the effect of camera motion on Optical Flow estimation.

Camera movement causes many pixels in the scene to move simultaneously.

For example, when the camera moves forward, objects may appear to expand outward from a point.

Therefore, optical flow contains both:

Object motion + Camera motion

To isolate object motion, techniques such as camera-motion compensation or background modeling can be used.

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

Five applications are:

Traffic monitoring Video surveillance Autonomous vehicles Human activity recognition Sports analytics

Other applications include robotics, drone navigation and video stabilization.

10. How can Optical Flow improve surveillance, robotics, and human activity recognition? Surveillance

It can detect unusual movement and track people or vehicles.

Robotics

It can help robots estimate movement and navigate their environment.

Human Activity Recognition

It can analyze body movement patterns such as:

Walking Running Falling Jumping Other activities

Therefore, optical flow provides useful motion information for intelligent vision systems.